

Sự Tiến Hóa của CI/CD Pipeline: Từ Fat CI tới GitOps với Vault Secret Management

Nghiên cứu thực nghiệm về các pattern triển khai và khuyến nghị áp dụng

Nghiên cứu nội bộ

May 19, 2026

Contents

Tóm tắt (Abstract)	2
Đặt vấn đề	2
Phần I: Vấn đề Fat CI và sự ra đời của Thin CI	2
1.1 Mô tả vấn đề	2
1.2 Hậu quả	3
1.3 Giải pháp: Thin CI, Thick Scripts	3
1.4 Lợi ích thực tế	4
Phần II: Vấn đề CD và giới hạn của Thin CI	4
2.1 Mô tả vấn đề	4
2.2 Các giải pháp	4
2.2.1 Multi-job pipeline với native CI features	4
2.2.2 CD tool chuyên dụng	5
2.2.3 Workflow engine có state	5
Phần III: Build Once, Deploy Anywhere	5
3.1 Mô tả vấn đề	5
3.2 Giải pháp	5
3.3 Hệ quả kiến trúc	5
Phần IV: GitOps và Source of Truth	6
4.1 Các trường phái CD chính	6
4.2 Anti-pattern phổ biến	6
4.3 Giải pháp đúng: Write-back vào Git	6
4.4 Cấu trúc repository khuyến nghị	7
4.5 Quy tắc vàng	7
Phần V: Quản lý Secret trong GitOps	7
5.1 Mô tả vấn đề	7
5.2 Hai trường phái giải quyết	8
5.2.1 Encrypted-in-Git	8
5.2.2 Centralized Vault	8
5.3 Đánh giá vận hành	8
5.4 Khuyến nghị theo quy mô	8

Phần VI: Test Repository	9
Phần VII: Khuyến nghị áp dụng	10
7.1 Lộ trình theo giai đoạn trưởng thành	10
7.2 Phân chia trách nhiệm	11
7.3 Các nguyên tắc bất biến	11
Kết luận	11
Tài liệu tham khảo	12

Tóm tắt (Abstract)

Tài liệu này khảo sát sự tiến hóa của các mô hình CI/CD pipeline trong môi trường phát triển phần mềm hiện đại, từ pattern “Fat CI” truyền thống cho tới mô hình GitOps kết hợp HashiCorp Vault cho quản lý secret tập trung. Bằng cách phân tích từng vấn đề cụ thể và các giải pháp tương ứng, nghiên cứu này đề xuất một lộ trình áp dụng phù hợp cho các tổ chức ở các giai đoạn trưởng thành kỹ thuật khác nhau. Trọng tâm của nghiên cứu là pattern “Thin CI, Thick Scripts” kết hợp với GitOps và “Build Once, Deploy Anywhere”, được chứng minh là phù hợp với 99% các tổ chức.

Đặt vấn đề

Trong quá trình vận hành CI/CD pipeline, các tổ chức thường gặp phải một loạt vấn đề có tính lặp lại theo thời gian. Mỗi vấn đề lại dẫn tới một giải pháp, và mỗi giải pháp lại tạo ra vấn đề mới. Nghiên cứu này khảo sát chuỗi tiến hóa đó theo trình tự thời gian phát sinh, đồng thời chỉ ra điểm cân bằng tối ưu cho từng loại tổ chức.

Các câu hỏi nghiên cứu cụ thể:

1. Làm thế nào để xây dựng CI/CD pipeline vừa portable, vừa dễ bảo trì?
2. Khi nào cần dùng nhiều CI provider, khi nào chỉ cần một?
3. Quản lý state production như thế nào trong môi trường có yêu cầu CD phức tạp (approval, canary, multi-region)?
4. Source of truth trong GitOps thực sự là gì, và xử lý anti-pattern ra sao?
5. Quản lý secret trong GitOps theo cách nào là an toàn và có khả năng mở rộng?

Phần I: Vấn đề Fat CI và sự ra đời của Thin CI

1.1 Mô tả vấn đề

Pattern Fat CI là cách triển khai phổ biến nhất khi một tổ chức mới bắt đầu với CI/CD. Toàn bộ logic build, test, và deploy được nhồi vào file cấu hình của CI provider (ví dụ `.github/workflows/deploy.yml`). Một pipeline deploy điển hình theo cách này có thể lên tới 70-80 dòng YAML, bao gồm:

- Checkout code

- Setup runtime (Python, Go)
- Install dependencies
- Run tests
- Build artifact
- Configure cloud credentials
- Determine deployment environment
- Upload artifact lên cloud storage
- Invalidate CDN cache
- Send notifications

1.2 Hậu quả

Pattern này dẫn tới các vấn đề sau:

Vendor lock-in nghiêm trọng: Khi tổ chức quyết định chuyển từ GitHub Actions sang GitLab CI (hoặc Jenkins, hoặc CodePipeline), toàn bộ pipeline phải được viết lại từ đầu với cú pháp mới.

Không thể debug local: Khi pipeline fail trên CI, developer phải push commit “fix CI” liên tục để thử nghiệm — không có cách nào reproduce lỗi trên máy local.

Khó code review: YAML logic phức tạp với điều kiện rẽ nhánh và biến môi trường khó review hơn nhiều so với code được viết bằng ngôn ngữ lập trình thông thường.

Onboarding chậm: Developer mới phải đọc hàng trăm dòng YAML rải rác để hiểu hệ thống deploy.

1.3 Giải pháp: Thin CI, Thick Scripts

Pattern Thin CI tách biệt rõ ràng giữa hai tầng:

Tầng 1 - CI provider (thin): Chỉ chịu trách nhiệm về trigger, secrets management, runner provisioning, và notification. File CI chỉ khoảng 5-10 dòng, gọi một lệnh duy nhất.

Tầng 2 - Scripts (thick): Chứa toàn bộ logic build/test/deploy thật sự. Được viết bằng bash/Python/Go, có thể chạy trên cả CI và máy local của developer.

Ví dụ minh họa với cùng một logic deploy:

```
# GitHub Actions
- run: ./scripts/deploy.sh production

# GitLab CI
script:
  - ./scripts/deploy.sh production

# Jenkins
sh './scripts/deploy.sh production'
```

```
# AWS CodeBuild
```

```
commands:
```

```
- ./scripts/deploy.sh production
```

Toàn bộ logic deploy nằm trong file `scripts/deploy.sh`, không bị phụ thuộc vào cú pháp của bất kỳ CI provider nào.

1.4 Lợi ích thực tế

Mặc dù Thin CI thường được giới thiệu với lợi ích “đổi CI provider dễ dàng”, trên thực tế giá trị quan trọng hơn nằm ở:

1. **Khả năng chạy local:** Developer có thể chạy `./scripts/deploy.sh staging` ngay trên máy mình để reproduce và debug.
2. **Testability:** Logic deploy là code, có thể được test như mọi code khác.
3. **Code review chất lượng:** Logic bằng bash/Python dễ review hơn YAML.
4. **Separation of concerns:** CI provider lo về infrastructure (trigger, secrets), script lo về logic.
5. **Onboarding nhanh:** Developer mới đọc một file script là hiểu toàn bộ hệ thống.

Phần II: Vấn đề CD và giới hạn của Thin CI

2.1 Mô tả vấn đề

Pattern Thin CI hoạt động tốt cho giai đoạn build và test. Tuy nhiên khi tới giai đoạn CD (Continuous Deployment), pipeline cần có khả năng tạm dừng để chờ các hành động bên ngoài:

- **Manual approval:** Quản lý sản phẩm cần phê duyệt trước khi deploy production
- **Smoke test wait:** Deploy staging xong, chờ 10 phút theo dõi metrics
- **Canary rollout:** Deploy 5% traffic, chờ 30 phút, sau đó 50%, rồi 100%
- **Change window:** Chỉ deploy production trong khung giờ cho phép
- **Rollback gate:** Tự động rollback nếu error rate vượt ngưỡng

Một bash script chạy tuần tự không phù hợp cho các yêu cầu này. Process CI tính tiền theo phút chạy, và nếu mất kết nối thì toàn bộ tiến trình bị hủy.

2.2 Các giải pháp

2.2.1 Multi-job pipeline với native CI features

Sử dụng jobs, needs, và environments của GitHub Actions/GitLab CI để chia pipeline thành nhiều stage có thể dừng được. Mỗi stage vẫn gọi một script portable, nhưng pipeline tổng thể được điều phối bởi CI provider.

2.2.2 CD tool chuyên dụng

Tách CD ra khỏi CI hoàn toàn. CI chỉ build và push artifact. CD được quản lý bởi một tool riêng:

- **Argo CD / Flux CD:** GitOps cho Kubernetes
- **Spinnaker:** Multi-region deployment với canary analysis
- **AWS CodeDeploy:** Blue/green và canary native
- **Octopus Deploy:** Enterprise CD với approval gates

2.2.3 Workflow engine có state

Với pipeline phức tạp chạy nhiều giờ và cần persist state qua restart:

- **Temporal.io:** Durable workflow execution
- **AWS Step Functions:** Serverless workflow orchestration
- **Argo Workflows:** K8s-native workflow engine

Phần III: Build Once, Deploy Anywhere

3.1 Mô tả vấn đề

Khi một tổ chức có nhiều môi trường (dev, staging, production) và nhiều region, một câu hỏi quan trọng phát sinh: build artifact ở mỗi môi trường, hay build một lần và deploy nhiều nơi?

Nếu build riêng cho mỗi môi trường: - Lãng phí tài nguyên CI - Nguy cơ artifact ở các môi trường khác nhau (do dependency floating, base image update) - Test ở staging không đảm bảo prod sẽ chạy đúng

3.2 Giải pháp

Nguyên tắc **Build Once, Deploy Anywhere** quy định:

1. CI pipeline chạy một lần, tạo ra một artifact duy nhất (thường là Docker image)
2. Artifact được push lên một registry duy nhất với version tag (ví dụ v1.2.3 hoặc commit-sha)
3. Mọi môi trường deploy từ cùng artifact đó
4. Không bao giờ build lại để rollback — chỉ deploy lại version cũ

Mô hình này đảm bảo artifact đã được test ở dev/staging chính là artifact sẽ chạy ở production. Không có khả năng “khác chút xíu” giữa các môi trường.

3.3 Hệ quả kiến trúc

Với nguyên tắc này, **đa số tổ chức không cần nhiều CI provider** — một CI duy nhất là đủ. Pattern thin CI vẫn giữ nguyên giá trị, nhưng giá trị chính không còn là “đổi provider dễ” mà là “logic deploy reproducible”.

Các trường hợp ngoại lệ cần nhiều CI provider:

1. **M&A**: Hai công ty sáp nhập với CI provider khác nhau
2. **Air-gapped + cloud hybrid**: Một số workload phải chạy trên môi trường không có internet
3. **Compliance theo region**: GDPR yêu cầu runner ở EU, US workload dùng CI khác
4. **Legacy migration in progress**: Jenkins chạy lâu năm, không thể chuyển một lần

Phần IV: GitOps và Source of Truth

4.1 Các trường phái CD chính

Nghiên cứu xác định bốn paradigm CD chính dựa trên câu hỏi “source of truth nằm ở đâu”:

Push-based CD: CI runner đẩy code/manifest lên server đích qua SSH, kubectl, hoặc cloud SDK. Source of truth là CI script. Đơn giản, phổ biến nhất ở startup. Nhược điểm: không có self-healing, drift detection yếu, audit trail hạn chế.

Pull-based CD (GitOps): Agent trên cluster watch Git repo, tự pull manifest về và apply. Source of truth là Git repo. Đại diện: Argo CD, Flux CD. Self-healing tự động, audit trail đầy đủ.

Image-tag watching: Watcher theo dõi registry, tự update deployment khi có tag mới. Source of truth là registry. Đại diện: Keel, Argo CD Image Updater (direct apply mode). Đơn giản nhưng audit kém.

Event-driven workflow: Pipeline phức tạp có state, xử lý bởi workflow engine. Source of truth là state engine. Đại diện: Spinnaker, Temporal, AWS Step Functions. Mạnh nhưng phức tạp, chỉ phù hợp enterprise.

4.2 Anti-pattern phổ biến

Một sai lầm thường gặp khi dùng Argo CD: cho Argo CD Image Updater watch registry trực tiếp với chế độ write-back-method: argocd. Cách này tạo ra hai source of truth:

- Git repo chứa manifest (replicas, env vars, config)
- Image registry chứa tag thật sự đang chạy

Hậu quả: Git không còn phản ánh state thật của cluster. Disaster recovery từ Git sẽ rollback ngược thời gian. Mất hết ưu điểm cốt lõi của GitOps.

4.3 Giải pháp đúng: Write-back vào Git

Argo CD Image Updater cần được cấu hình với write-back-method: git. Khi phát hiện image tag mới, Image Updater commit vào Git repo, sau đó Argo CD sync như

bình thường từ Git. Git vẫn là source of truth duy nhất.

Có ba pattern phổ biến để cập nhật image tag vào Git:

1. **CI tự commit**: CI build xong, tự commit thay đổi vào manifest repo (đơn giản nhất, phổ biến nhất)
2. **Argo CD Image Updater với git write-back**: Tách concern giữa CI và CD
3. **Flux Image Automation**: Tương tự cho người dùng Flux CD

4.4 Cấu trúc repository khuyến nghị

Pattern hai repository tách biệt:

```
application-repo/                                # Code repository
├── src/
├── Dockerfile
└── .github/workflows/ci.yml

manifest-repo/                                    # Argo CD watches this
├── apps/
│   └── my-app/
│       ├── base/
│       └── overlays/
│           ├── dev/values.yaml
│           ├── staging/values.yaml
│           └── prod/values.yaml
```

Rollback trong mô hình này chỉ cần git revert trong manifest repo. Argo CD tự sync version cũ trong vòng 30 giây. Không cần CI chạy lại, không cần build lại.

4.5 Quy tắc vàng

Trong GitOps, mọi thay đổi production phải đi qua Git commit. UI của Argo CD chỉ để observe state, không phải để modify state. Nếu một thay đổi không có dấu vết trong Git, nó không tồn tại — và sẽ bị Argo CD revert về Git khi sync chu kỳ tiếp theo.

Phần V: Quản lý Secret trong GitOps

5.1 Mô tả vấn đề

GitOps yêu cầu mọi state nằm trong Git. Nhưng secret cần được giữ kín, trong khi Git được thiết kế để chia sẻ. Đây là mâu thuẫn cơ bản cần giải quyết.

Plaintext secret trong Git là sai lầm nghiêm trọng:

- Git history forever — xóa file sau cũng vẫn còn trong history
- Mọi developer clone repo đều thấy
- Backup, mirror, fork đều nhân bản secret

- Đã có nhiều vụ leak secret qua public repository

5.2 Hai trường phái giải quyết

5.2.1 Encrypted-in-Git

Secret được mã hóa trước khi commit. Chỉ cluster mới có key giải mã.

Sealed Secrets (Bitnami): Mã hóa bằng public key của cluster, controller trong cluster giải mã với private key. Đơn giản, miễn phí, phù hợp với GitOps. Nhược điểm: quản lý key rotation phức tạp, mỗi environment cần bộ key riêng, không có audit “ai đã đọc secret”.

5.2.2 Centralized Vault

Secret nằm hoàn toàn ngoài Git. Git chỉ chứa reference tới nơi lưu secret thật.

HashiCorp Vault: Trung tâm hóa secret management. Vault Agent Injector inject secret trực tiếp vào pod qua sidecar. Hỗ trợ dynamic secrets (credentials sinh ra theo yêu cầu với TTL ngắn), auto-rotation, audit log đầy đủ, và fine-grained access policy. Tích hợp với mọi cloud provider thông qua các secret engine.

5.3 Đánh giá vận hành

Trải nghiệm thực tế cho thấy Sealed Secrets có chi phí vận hành cao khi scale:

- Onboarding developer mới mất 30 phút setup kubeseal CLI, lấy cert, học workflow
- Key rotation phức tạp, mất key đồng nghĩa mất toàn bộ secret
- Mỗi environment một bộ key riêng, mỗi lần đổi secret phải encrypt 3 lần
- Không có audit “ai đọc secret”
- Không có dynamic secrets / auto rotation

Vault giải quyết toàn bộ các vấn đề trên, đồng thời cung cấp dynamic secrets và tích hợp tự nhiên với cloud secret services (AWS Secrets Manager, GCP Secret Manager) thông qua các secret engine.

5.4 Khuyến nghị theo quy mô

Quy mô tổ chức	Khuyến nghị
Startup, ít secret, không có Vault	Sealed Secrets
Trung bình trở lên, có K8s	HashiCorp Vault với Vault Agent Injector
Enterprise multi-cloud, compliance cao	Vault tập trung với dynamic secrets

Phần VI: Test Repository

Để kiểm chứng các pattern được đề xuất, một test repository sẽ được tạo với cấu trúc sau:

```
cicd-evolution-demo/
├── README.md                                # Mô tả mục tiêu và cách chạy
├── application/                             # Application code repository
│   ├── src/
│   │   └── app.py                          # Sample Python application
│   ├── tests/
│   │   └── test_app.py
│   ├── Dockerfile
│   ├── scripts/                            # Thick scripts (portable logic)
│   │   ├── build.sh
│   │   ├── test.sh
│   │   ├── deploy.sh
│   │   └── lib/
│   │       └── common.sh
│   └── ci-providers/                       # Thin CI configs
│       ├── github-actions.yml              # Demonstrates thin CI
│       ├── gitlab-ci.yml                  # Same logic, different provider
│       ├── jenkins/Jenkinsfile
│       └── codebuild/buildspec.yml
├── manifests/                              # GitOps manifest repository
│   ├── apps/
│   │   ├── sample-app/
│   │   │   ├── base/
│   │   │   │   ├── deployment.yaml
│   │   │   │   ├── service.yaml
│   │   │   │   └── kustomization.yaml
│   │   │   └── overlays/
│   │   │       ├── dev/
│   │   │       ├── staging/
│   │   │       └── prod/
│   └── argocd/
│       ├── applications/                  # Argo CD Application CRDs
│       └── projects/
├── secrets-examples/
│   ├── sealed-secrets/                   # Example: Encrypted-in-Git
│   └── vault/                             # Example: Vault Agent Injector
└── docs/
    ├── fat-vs-thin-ci.md                 # Compare both patterns
    ├── gitops-setup.md
    └── secret-management.md
```

Test repository này sẽ minh họa cụ thể:

1. So sánh Fat CI (.github/workflows/fat-ci-example.yml) với Thin CI (scripts/deploy.sh + minimal CI config)
2. Pattern Build Once Deploy Anywhere với một image tag đi qua dev → staging → prod
3. GitOps workflow với Argo CD, bao gồm cả image updater write-back
4. Hai pattern quản lý secret khác nhau cho so sánh

Một developer có thể clone repo, chạy local với ./scripts/deploy.sh dev, và quan sát toàn bộ vòng đời artifact từ code commit tới deploy production.

Phần VII: Khuyến nghị áp dụng

7.1 Lộ trình theo giai đoạn trưởng thành

Giai đoạn 1 - Startup (1-10 developer):

- CI provider: chọn một (GitHub Actions hoặc GitLab CI)
- Pattern: Thin CI + Thick Scripts
- CD: Push-based, script trực tiếp deploy
- Secret: K8s Secret tạo thủ công, document cẩn thận
- Cấu trúc: Một repo duy nhất

Tránh over-engineering. Argo CD, Vault, Spinnaker không cần thiết ở giai đoạn này.

Giai đoạn 2 - Tăng trưởng (10-50 developer):

- Bắt đầu áp dụng GitOps với Argo CD
- Tách application repo và manifest repo
- Build Once, Deploy Anywhere với Docker registry
- Secret: Sealed Secrets, hoặc bắt đầu với Vault nếu đã có platform team
- CD: Multi-stage pipeline với manual approval cho production

Giai đoạn 3 - Scale (50+ developer):

- Platform team xây Vault tập trung phục vụ toàn bộ K8s cluster
- Vault Agent Injector cho dynamic secrets và auto-rotation
- Progressive Delivery với Argo Rollouts hoặc Flagger (canary tự động)
- Feature flags để tách deploy và release
- Multi-cluster Argo CD nếu deploy đa region

Giai đoạn 4 - Enterprise (100+ developer):

- Workflow engine cho CD phức tạp (Temporal, Spinnaker)
- Multi-region, multi-cloud, multi-account architecture
- Compliance automation (audit, change approval)
- Dedicated SRE team

7.2 Phân chia trách nhiệm

Mô hình platform team / product team được khuyến nghị mạnh:

Platform team xây dựng và vận hành: Argo CD cluster, Vault, monitoring stack, base CI/CD templates, shared scripts library.

Product team sử dụng: viết application code, manage manifest cho service của mình, định nghĩa Argo CD Application, không cần biết về internal của Vault hay Argo CD.

Phân chia này tránh được tình trạng mọi developer phải học toàn bộ stack DevOps để deploy được một feature.

7.3 Các nguyên tắc bất biến

Bất kể quy mô tổ chức, năm nguyên tắc sau luôn áp dụng:

1. **Logic deploy phải chạy được local.** Nếu không, không phải logic — là magic.
2. **Build một artifact, deploy nhiều nơi.** Không bao giờ build riêng cho từng môi trường.
3. **Mọi thay đổi production phải qua Git commit.** UI là để xem, không phải để sửa.
4. **Không bao giờ plaintext secret trong Git.** Kể cả “tạm thời”.
5. **Rollback phải đơn giản và nhanh.** `git revert` thắng `kubectl rollout undo`.

Kết luận

Sự tiến hóa của CI/CD pipeline phản ánh quá trình các tổ chức học hỏi từ những đau đớn vận hành cụ thể. Pattern Thin CI giải quyết vấn đề vendor lock-in và testability. GitOps giải quyết vấn đề audit trail và self-healing. HashiCorp Vault giải quyết vấn đề chi phí vận hành của secret encryption và cung cấp dynamic secrets cho quy mô lớn.

Tuy nhiên, không có pattern nào là “đúng tuyệt đối” — mỗi pattern phù hợp với một giai đoạn trưởng thành cụ thể của tổ chức. Sai lầm phổ biến là áp dụng pattern enterprise cho startup (over-engineering) hoặc giữ pattern startup khi đã scale (technical debt).

Đề xuất nghiên cứu tiếp theo:

1. Tích hợp Progressive Delivery (Argo Rollouts, Flagger) với GitOps
2. Feature flags như paradigm tách biệt deploy và release
3. Database migration trong GitOps context (Expand-Contract pattern)
4. Cost analysis: chi phí vận hành của các pattern khác nhau ở các scale khác nhau

Tài liệu tham khảo

1. Argo CD Documentation, <https://argo-cd.readthedocs.io>
2. Argo CD Image Updater, <https://argocd-image-updater.readthedocs.io>
3. HashiCorp Vault Documentation, <https://developer.hashicorp.com/vault>
4. Sealed Secrets, <https://github.com/bitnami-labs/sealed-secrets>
5. GitOps Principles, OpenGitOps, <https://opengitops.dev>
6. Twelve-Factor App methodology, <https://12factor.net>
7. Kubernetes documentation on Secrets, <https://kubernetes.io/docs/concepts/configuration/s>